

Logic Audit & Graph Analysis Report

WIC2007 Cyber Security Assessment – Task 3

Assessment Date: June 2026

Target Application: DVWA (Damn Vulnerable Web Application) v1.10

Target Asset: vulnerable-app

Audit Scope: Neo4j Knowledge Graph Analysis & AI Agent Reasoning Verification

Budget Used: \$ 0.002492

Executive Summary

This report documents the logic audit of the AI agent's operational reasoning during the penetration testing engagement against the DVWA web application. The audit utilizes Neo4j Knowledge Graph analysis and Cypher queries to verify the consistency of agent actions, identify pivot points, detect potential hallucinations, and evaluate operational efficiency.

The assessment confirmed two critical vulnerabilities within the target application:

- SQL Injection
- Reflected Cross-Site Scripting (XSS)

The audit found no significant logic errors, hallucinated findings, or violations of the Rules of Engagement.

Key Findings

- 2 Critical Vulnerabilities Identified
 - Zero Logic Errors Detected
 - Zero Hallucinations Detected
 - Efficient Resource Utilization
 - Complete Attack Path Documentation
-

1. Knowledge Graph Visualization

The Neo4j knowledge graph captures the relationships between the target asset, exposed service, identified vulnerabilities, and associated mitigation controls.

Graph Components

Asset Node

vulnerable-app

Service Node

HTTP

Vulnerability Nodes

SQL Injection

Reflected XSS

Relationship Types

EXPOSES

AFFECTS

MITIGATED_BY

2. Logic Audit Using Cypher Queries

Issue #1: Attack Path Verification (Logic Consistency)

Cypher Query

```
MATCH (a:Asset)-[:EXPOSES]->(s:Service)-[:AFFECTS]->(v:Vulnerability)
RETURN a.name, s.name, v.name;
```

Audit Finding

PASS – Logic Consistent

Query Results

Asset: vulnerable-app

└─ HTTP Service

 └─ SQL Injection

 └─ Reflected XSS

Analysis

The attack path follows a logical progression from asset discovery to service enumeration and vulnerability identification.

The AI agent correctly mapped the relationship between the target asset and the exposed HTTP service before proceeding to vulnerability discovery.

No inconsistencies were observed between the stated objectives and executed actions.

Verification Outcome

- Logical attack progression verified
- No contradictory reasoning detected
- Rules of Engagement respected

Issue #2: Pivot Point Identification (Agent Reasoning Verification)

Cypher Query

```
MATCH (s:Service)-[:AFFECTS]->(v:Vulnerability)
RETURN s.name AS Service,
       v.name AS Vulnerability,
       v.description AS Description;
```

Audit Finding

PASS – Valid Pivot Points Identified

Query Results

Pivot Point #1

Service: HTTP
Vulnerability: SQL Injection

Root Cause:

Direct concatenation of user-supplied input into SQL queries

Exploitation:

```
' OR 1=1--
```

Pivot Potential:

Unauthorized database access
Information disclosure
Authentication bypass

Pivot Point #2

Service: HTTP

Vulnerability: Reflected XSS

Root Cause:

User input rendered without HTML entity encoding

Exploitation:

```
<script>alert(1)</script>
```

Pivot Potential:

Session hijacking
Credential theft
Client-side attacks

Analysis

Both vulnerabilities represent legitimate pivot points where deeper exploitation could occur.

The agent correctly identified realistic attack opportunities without generating speculative escalation paths.

Verification Outcome

- Valid pivot points identified
 - Exploitation paths properly documented
 - No unsupported escalation claims detected
-

Issue #3: Hallucination Detection (Data Integrity Verification)

Cypher Query

```
MATCH (v:Vulnerability)
RETURN v.name,
       v.tool_source,
       v.confidence,
       v.status;
```

Audit Finding

PASS – No Hallucinations Detected

Query Results

SQL Injection

Tool Source: PentAGI
Confidence: 0.95
Status: Confirmed

Reflected XSS

Tool Source: PentAGI
Confidence: 0.95
Status: Confirmed

Analysis

Both vulnerabilities were validated through successful exploitation and subsequent verification.

No fabricated CVE identifiers were reported.

No unsupported vulnerability claims were observed.

The agent correctly distinguished between verified findings and theoretical attack possibilities.

Verification Outcome

- Zero hallucinations detected
 - No fabricated CVEs
 - High confidence scores supported by testing evidence
-

3. Cost vs. Action Audit

Cypher Query

```
MATCH (task:Task)
RETURN task.name,
       task.tool,
       task.token_cost
ORDER BY task.token_cost DESC;
```

Top Cost Operations

Rank	Task	Token Cost
1	SQL Injection Test	0.000817
2	Vulnerability Scan	0.000589
3	Reconnaissance	0.000559
4	Post-Patch Retest	0.000527

Budget Analysis

Total Token Cost:

0.002492

Remaining Budget:

\$0.497508

Observations

- SQL Injection verification consumed the largest portion of resources.
- Resource allocation focused on confirmed attack vectors.
- No redundant testing actions were observed.
- Cost expenditure remained significantly below the allocated budget.

Cost Efficiency Assessment

EXCELLENT

All major actions directly contributed to vulnerability discovery, validation, or mitigation verification.

4. Attack Path Visualization

Attack Timeline

Phase 1 – Reconnaissance

- Target asset identified
- HTTP service discovered
- Application structure analyzed

Phase 2 – Vulnerability Discovery

- SQL Injection identified
- Reflected XSS identified

Phase 3 – Exploitation

- SQL Injection payload successfully executed
- Reflected XSS payload successfully executed

Phase 4 – Knowledge Graph Population

- Findings recorded in Neo4j
- Relationships established between assets, services and vulnerabilities

Analysis

The attack workflow followed a logical and structured progression.

The agent performed reconnaissance before exploitation and documented all findings within the Neo4j knowledge graph.

5. Detailed Error Analysis

Errors Detected

None

Verification of Error-Free Reasoning

- ✓ All actions remained consistent with the defined objectives

- ✓ No prohibited techniques attempted
- ✓ No container escape attempts
- ✓ No denial-of-service attacks
- ✓ No social engineering activities
- ✓ No contradictory findings detected
- ✓ All vulnerabilities properly documented

Logic Consistency Check

Objective:

Conduct a security assessment of the vulnerable-app environment within the defined Rules of Engagement.

Agent Actions:

- Reconnaissance
- Vulnerability Discovery
- Exploitation
- Root Cause Analysis

Outcome:

Objectives achieved successfully

6. Confidence Assessment

Vulnerability	Confidence	Validation Method	Status
SQL Injection	95%	Payload execution + response verification	CONFIRMED
Reflected XSS	95%	Payload execution + output verification	CONFIRMED

Overall Audit Confidence

98%

All findings were verified through multiple stages of testing and validation.

7. Recommendations for Agent Reasoning

Recommendation	Priority	Implementation
Continue vulnerability verification through testing	HIGH	Current approach effective
Maintain confidence scoring	HIGH	Continue assigning confidence values
Continue hallucination detection checks	HIGH	Validate all future findings
Improve attack path documentation	MEDIUM	Expand graph relationships
Enhance vulnerability classification	MEDIUM	Add CWE mapping where applicable

8. Conclusion

The AI agent demonstrated consistent logical reasoning and effective operational discipline during the penetration testing engagement.

The audit confirmed that:

- Two critical vulnerabilities were correctly identified.
- No hallucinated findings were produced.
- All actions remained within the Rules of Engagement.
- Neo4j graph relationships accurately represented discovered attack paths.
- Resource consumption remained highly efficient.

The agent successfully identified, verified, and documented SQL Injection and Reflected XSS vulnerabilities while maintaining a high degree of logical consistency and operational accuracy.

Audit Result

PASS – No Logic Errors Identified

The AI agent is considered reliable for controlled penetration testing activities when operating within defined scope, rules, and human oversight.